

1. Implemented Technology Stack

Layer	Technology	Role
UI	Streamlit	Interactive form, preview, editing and downloads
API	FastAPI + Uvicorn	Internal HTTP service for generation and health endpoints
Validation	Pydantic	Request and response schema validation
AI	Google GenAI SDK + Gemini	Legal-draft generation
Formatting	python-docx + fpdf2	DOCX and PDF generation
Testing	pytest + FastAPI TestClient	Regression and API testing
Config	python-dotenv	Load environment variables from .env

2. Core Implementation

The application is split into an internal FastAPI service and a Streamlit interface. The UI sends a JSON payload containing document_type, parties, terms and dates. The backend returns the generated document and metadata such as demo_mode and model used.

3. AI Generation Strategy

- Prompt construction is centralized in GeminiDocumentGenerator.build_prompt().
- The prompt instructs the model to use only supplied information and square-bracket placeholders for missing material facts.
- Model selection can inspect accessible generation-capable models when the provider permits listing.
- Configured candidates are attempted in order, and temporary 429/503-style provider failures receive retries before fallback.
- DEMO_MODE bypasses live generation and produces a deterministic sample document for demonstrations.

4. Frontend Workflow

Stage	Implemented behavior
Input	Select document type; enter parties, dates and semicolon-separated terms.
Submit	Client-side missing-field check, then POST /generate.
Preview	Render headings, lists, numbered clauses, bold, italic and code safely.
Edit	Generated content can be reviewed and modified in the workspace.
Export	Download TXT, DOCX or branded PDF; DOCX also includes a terms summary table.

5. File-Level Development Evidence

File	Observed implementation
backend/routes.py	/, /health and POST /generate endpoints
backend/schemas.py	DocumentRequest and DocumentResponse validation models
backend/ai_core/gemini_generator.py	Prompting, model candidate selection, retries, fallback and demo document
backend/utils/text.py	Sanitization, terms parsing, filename safety and Markdown rendering
backend/services/docx_formatter.py	Professional DOCX formatting and terms table

File	Observed implementation
backend/services/pdf_formatter.py	PDF formatter
frontend/app.py	Workspace UI, state handling, preview, edit and downloads

6. Running the Project

Backend

```
python -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend

```
python -m streamlit run frontend/app.py
```

Tests

```
python -m pytest -q
```

Observed test execution on the supplied source package: 14 passed in 0.72s.